



SAPIENZA
UNIVERSITÀ DI ROMA

Sapienza-DC: progettazione e sviluppo di un sistema RAG domain-aware

Implementazione di un chatbot multi-dominio basato su Retrieval-Augmented Generation

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica (I3S)
Dipartimento di Ingegneria Informatica Automatica e Gestionale (DIAG)
Corso di Laurea in Ingegneria Informatica e Automatica

Fabio Pastore

Matricola 2129632

Relatore

Prof. Roberto Navigli

Anno Accademico 2025/2026

Sapienza-DC: progettazione e sviluppo di un sistema RAG domain-aware

Tesi di laurea triennale. Sapienza Università di Roma

© 2026 Fabio Pastore. Tutti i diritti riservati

Questa tesi è stata composta con L^AT_EX e la classe Sapthesis.

Email dell'autore: pastore.2129632@studenti.uniroma1.it

To my family and friends.

Sommario

Nella seguenti tesi viene descritta la realizzazione di Sapienza-DC, un chatbot RAG multi-dominio capace di rispondere alle richieste degli utenti mediante analisi e validazione di informazioni estratte da pagine web, fornendo risposte coerenti affiancate da un punteggio di affidabilità nonché dalle fonti utilizzate dall’LLM per rispondere.

I capitoli contenuti in questo elaborato tratteranno gli obiettivi fondamentali di Sapienza-DC e la raccolta dei requisiti di progetto (Cap. I); l’analisi delle tecnologie nonché metodologie utilizzate (Cap. II), la progettazione (Cap. III), l’architettura del sistema (Cap. IV), la fase di implementazione e gli sviluppi del progetto, proposti in ordine cronologico (Cap. V); un breve excursus sulle ottimizzazioni introdotte, la sicurezza della pipeline dinanzi a tentativi di prompt injection e una valutazione qualitativa, sotto forma di benchmark, del sistema complessivo (Cap. VI). Al termine della trattazione verranno invece effettuate alcune considerazioni sul lavoro svolto, sui traguardi raggiunti e su possibili sviluppi del progetto (Cap. VII).

Indice

1	Introduzione	1
1.1	Presentazione del progetto	1
1.2	Requisiti di progetto	1
2	Analisi delle tecnologie e metodologie impiegate	3
2.1	Concetti preliminari di NLP	3
2.1.1	Large Language Model (LLM)	4
2.1.2	Retrieval-Augmented Generation (RAG)	4
2.1.3	Bi-encoder, cross-encoder	4
2.2	Framework, strumenti e librerie di sviluppo	6
2.2.1	FastAPI	6
2.2.2	LangChain	6
2.2.3	Hugging Face	6
2.2.3.1	FastEmbed	7
2.2.3.2	SentenceTransformers	7
2.2.4	Ollama e llama.cpp	7
2.2.5	SearXNG	8
2.2.6	Crawl4AI	9
2.2.7	Jinja2	9
2.2.8	TailwindCSS	9
2.2.8.1	Utility class	9
2.3	Containerizzazione ed orchestrazione	9
2.3.1	Architettura a microservizi: Docker e Docker Compose	10
2.4	Database relazionale	10
2.4.1	SQL	10
2.4.2	MariaDB	11
2.5	Metodologia di sviluppo agile	11
3	Progettazione dell'applicazione	13
3.1	Diagramma delle classi UML	13
3.2	Modello ER	15
3.2.1	Progettazione concettuale	15
3.2.2	Progettazione logica	16
3.2.2.1	Ristrutturazione dello schema concettuale	16
3.2.2.2	Traduzione diretta	17
3.2.2.3	Ristrutturazione dello schema logico	17

4	Architettura del sistema	19
4.1	Architettura lato server (backend)	20
4.2	Architettura lato client (frontend)	21
4.3	Architettura per la persistenza dei dati (MariaDB)	22
4.4	Architettura di integrazione ed estrazione dal web (MWP)	22
5	Implementazione della pipeline RAG	23
6	Ottimizzazioni, sicurezza e benchmark	25
7	Conclusione	27
	Bibliografia	29

Capitolo 1

Introduzione

1.1 Presentazione del progetto

Sapienza-DC [6] è stato realizzato come progetto finale per il corso di Laboratorio di Ingegneria Informatica.

In seguito alla realizzazione di un progetto preliminare, il quale consisteva nella realizzazione di un parser per opportuni domini assegnati a ciascun gruppo partecipante, sono stati selezionati i migliori progetti, al fine di avere l'opportunità di accedere ad un progetto personalizzato.

Tra i vari progetti personalizzati veniva proposta la realizzazione di un chatbot di dominio capace di ricevere in input una query da parte dell'utente, selezionare un dominio su cui cercare informazioni per rispondere alla domanda, ricercare fonti su pagine web appartenenti a quest'ultimo, estrarre e processare i contenuti rilevanti da fornire a un LLM avente il compito di rispondere al quesito posto, fornendo aggiuntivamente le fonti di riferimento impiegate.

1.2 Requisiti di progetto

Le specifiche fornite preliminarmente per la realizzazione del progetto consistevano in una descrizione generale dell'architettura della pipeline da implementare:

- (i) consentire all'utente di porre al proprio sistema una query;
- (ii) assicurarsi che la query dell'utente possa essere risolta sfruttando i domini disponibili;
- (iii) ricercare online pagine (nei domini disponibili) che possano contenere informazioni utili;
- (iv) parsare le pagine (sfruttando i parser dell'esonero) e fornire ad un LLM la query assieme ai dati estratti da queste ultime;
- (v) restituire la risposta all'utente.

In aggiunta a quanto appena elencato, si richiedevano inoltre diverse funzionalità aggiuntive. In particolare, il sistema implementato sarebbe dovuto essere capace di:

gestire richieste ambigue, chiedendo, ove necessario, chiarimenti all'utente; rigettare la query dell'utente qualora quest'ultima fosse ritenuta inaccettabile o inopportuna; classificare i contenuti delle pagine web per grado di pertinenza alla domanda, valutando eventualmente la possibilità di estrarre dati da domini qualunque in caso di query esterna ai domini assegnati; riassumere o selezionare porzioni delle informazioni ottenute in fase di parsing per evitare di saturare la context window del large language model; citare le fonti e stimare l'affidabilità della risposta fornita, gestendo opportunamente il caso in cui le informazioni in possesso fossero parziali o insufficienti per rispondere alla domanda dell'utente.

Di seguito viene proposto il diagramma utilizzato per presentare i requisiti di progetto.

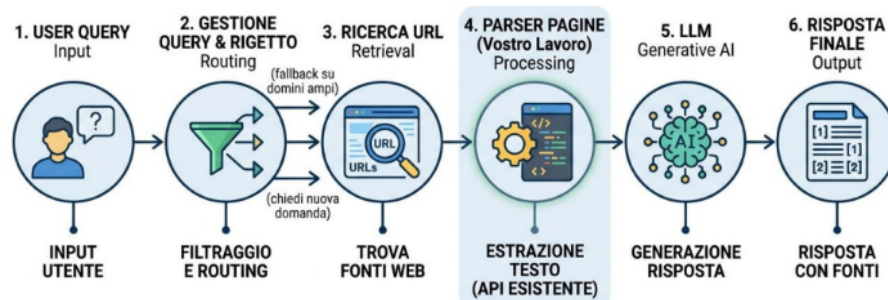


Figura 1.1. Diagramma esemplificativo fornito assieme ai requisiti di progetto.

Si noti come tali specifiche rappresentino solo una linea guida alla realizzazione del progetto, descrivendo solo gli aspetti generali e le funzionalità fondamentali che quest'ultimo dovrebbe implementare.

Questo è voluto: tra gli obiettivi del progetto personalizzato vi è proprio quello di lasciare al gruppo la libertà, e dunque anche le difficoltà che ne conseguono, di produrre quanto richiesto utilizzando gli strumenti nonché tecnologie ritenute più consone.

A tal riguardo, nel capitolo che segue introdurremo le principali tecnologie e framework utilizzati per sviluppare Sapienza-DC.

Capitolo 2

Analisi delle tecnologie e metodologie impiegate

In questo capitolo verranno fornite informazioni inerenti alle principali tecnologie e metodologie utilizzate durante lo sviluppo di Sapienza-DC, in modo tale da chiarire il ruolo e il funzionamento di ciascuno strumento impiegato durante la fase di implementazione del progetto.

2.1 Concetti preliminari di NLP

L’NLP (Natural Language Processing) è una branca dell’Intelligenza Artificiale che studia la lettura, comprensione, interpretazione nonché generazione, da parte di un calcolatore, di linguaggio umano, in modo naturale.

NLP combina in un unicum la linguistica computazionale, ossia le regole del linguaggio, con tecniche avanzate di machine learning¹ e deep learning². In una tipica pipeline NLP, il testo viene dapprima elaborato mediante tokenizzazione, la quale consiste nella suddivisione del testo in unità più piccole, quali parole o frasi. Dopodiché, ciascun token viene standardizzato, mediante conversione in minuscolo di ciascun carattere, la rimozione delle cosiddette stop word (parole del linguaggio comune utilizzate frequentemente, come "è", "la", etc.) nonché caratteri speciali o di punteggiatura, considerati come rumore in questa fase iniziale.

Successivamente, ciascun token viene trasformato, mediante opportune tecniche di NLP, in rappresentazioni numeriche su cui un calcolatore può operare. Ad esempio, tramite opportuni modelli di embedding (incorporamento) è possibile trasformare un token in un vettore n-dimensionale di reali, ove è possibile memorizzare il significato semantico del token originale. Questo, peraltro, sarà proprio l’approccio utilizzato da Sapienza-DC, come vedremo più in avanti.

A partire dai dati così ottenuti, vengono estratte ulteriori informazioni mediante opportune tecniche computazionali, al fine di identificare ruoli grammaticali delle

¹ machine learning: branca dell’IA avente come obiettivo quello di apprendere autonomamente da un insieme di dati in modo tale da essere in grado successivamente di effettuare previsioni su nuovi dati.

² deep learning: sottoinsieme del machine learning in cui si utilizzano reti neurali multistrato costituite da neuroni artificiali interconnessi.

parole, argomenti e sentiment, vale a dire il tono emotivo del testo. Infine, le nozioni raccolte vengono utilizzate per addestrare modelli di machine learning, capaci di apprendere tali relazioni e utilizzarle una volta completato l'addestramento per generare linguaggio naturale.

2.1.1 Large Language Model (LLM)

Un LLM è un particolare tipo di modello linguistico capace di generare linguaggio naturale mediante addestramento su una grande quantità di dati. Il termine "large" deriva dal grande (spesso nell'ordine di miliardi) numero di parametri del modello probabilistico appresi in fase di training. Gli LLM sono basati su deep learning e utilizzano come componenti fondamentali reti neurali e trasformatori³ (transformers).

Un LLM genera testo in modo puramente statistico, infatti, se $(w_1, w_2, \dots, w_T)$ sono parole che costituiscono una frase f , allora la probabilità che l'LLM $\mathcal{L}$ generi f è data da:

$$\mathbb{P}(w_1, w_2, \dots, w_T) = \prod_{j=1}^T \mathbb{P}(w_j | w_1, w_2, \dots, w_{j-1})$$

dove la probabilità di generare la j -esima parola w_j dipende dai parametri appresi durante l'addestramento del modello. La generazione, detta autoregressiva, avviene un token alla volta ed è pertanto sequenziale: ciascun nuovo token dipenderà direttamente da quelli generati in precedenza. Vi sarebbero numerosissime tematiche da approfondire a tal riguardo, ma, per brevità, considereremo sufficienti queste nozioni intuitive.

2.1.2 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation, ossia la generazione potenziata mediante recupero di dati, è un approccio che consiste nell'ottimizzazione dell'output di un LLM.

RAG prevede l'estrazione di informazioni e conoscenze utili da fonti autorevoli, al fine di accrescere le nozioni possedute dal modello. Tale strategia diviene estremamente utile quando è necessario che l'LLM fornisca una risposta a query altamente specifiche o inerenti a fatti recenti, altrimenti non contenuti nel dataset di addestramento dell'LLM.

Questa tecnica è adottata da Sapienza-DC per estrarre contenuti ed informazioni domain-specific con lo scopo di generare una risposta alle query proposte al chatbot da parte dell'utente.

2.1.3 Bi-encoder, cross-encoder

Un bi-encoder (o dual-encoder) è un modello utilizzato per convertire testo in vettori di reali, detti embedding dell'informazione di partenza. Il nome deriva dall'utilizzo di due reti neurali separate aventi lo scopo di trasformare rispettivamente query e documenti di riferimento nei relativi embedding.

³ trasformatore: particolare architettura di rete neurale artificiale che permette l'elaborazione parallela di diverse porzioni di testo, cogliendo inoltre eventuali relazioni tra parole distanti nel testo grazie a un meccanismo noto come self-attention.

⁴ Fonte immagine: obot.ai. Rilasciata sotto la seguente licenza: CC BY 4.0.

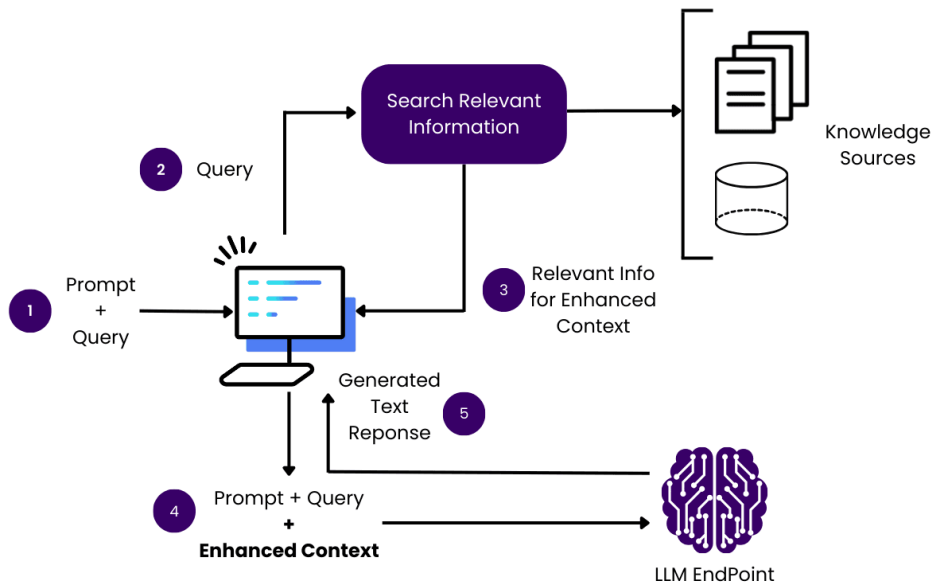


Figura 2.1. Elementi fondamentali di uno schema di RAG.⁴

L'idea a questo punto è molto semplice: verificare, mediante opportune operazioni sui vettori così ottenuti, quali documenti, o porzioni di essi, risultino più semanticamente affini alla query fornita dallo user. Un approccio utile a questo scopo che vedremo successivamente consisterà nel calcolo della cosine similarity tra v_q e v_c , vettori rispettivamente della query q e del chunk c , appartenente a un determinato documento ottenuto dalla corrispondente pagina web.

I bi-encoder sono notoriamente veloci, tuttavia, analizzando separatamente query e chunk dei documenti non godono di un'accuratezza eccellente. Per questo motivo è necessario utilizzare anche i cross-encoder. Questi ultimi sono modelli aventi lo stesso identico scopo, ma che offrono una precisione molto maggiore dei dual-encoder, a discapito di un maggiore costo computazionale.

In particolare, i cross-encoder devono necessariamente elaborare contemporaneamente query e chunk di testo, sfruttando i meccanismi di attenzione offerti dai transformer affinché sia colto il contesto della domanda e la reale pertinenza tra quest'ultima e la porzione di documento sotto esame. L'output di un cross-encoder non consiste in un vettore, ma in uno score, indice del grado di rilevanza del chunk c rispetto alla query q .

Nella fase di implementazione, utilizzeremo i cross-encoder, in seguito a un filtraggio preliminare mediante bi-encoder e cosine similarity, per effettuare reranking (riordinamento) dei chunk vari chunk estratti dalle fonti di interesse.

2.2 Framework, strumenti e librerie di sviluppo

2.2.1 FastAPI

FastAPI [8] è un framework web moderno, open source e ad alte prestazioni, progettato per la sviluppo di API⁵ (Application Programming Interface) REST (REpresentational State Transfer) in Python.

Rilasciato nel 2018, FastAPI è ben presto divenuto uno degli standard per lo sviluppo backend in Python, grazie alla sua efficienza nonché facilità di utilizzo.

Il framework usufruisce di due librerie fondamentali quali Starlette, un toolkit web ASGI (Asynchronous Server Gateway Interface) per la gestione di servizi asincroni, e Pydantic, grazie a cui è possibile una validazione automatica e personalizzabile dei dati e la creazione di modelli Pydantic per la definizione dei tipi di input e output di ciascun endpoint⁶.

Tali funzionalità facilitano e rendono estremamente intuitivo lo sviluppo di backend, garantendo prestazioni equiparabili ad altre tecnologie popolari come Node.js per lo sviluppo server-side.

Tra le funzionalità più apprezzate di FastAPI risalta la generazione automatica della documentazione API. Ogni qualvolta viene definito un nuovo endpoint, FastAPI genera automaticamente documentazione e interfacce Web UI (Swagger UI) mediante cui inviare richieste ai fini, ad esempio, di testing dell'applicazione.

Infine, il framework segue e adotta i principali standard web moderni per lo sviluppo di API basate su HTTP (HyperText Transfer Protocol), quali OpenAPI e JSON Schema per la validazione nonché serializzazione dei dati scambiati.

2.2.2 LangChain

LangChain [12] è un framework open source progettato per facilitare lo sviluppo di applicazioni che fanno uso di LLM.

Gli impieghi possibili di LangChain sono numerosi, ma ai fini dello sviluppo di Sapienza-DC ci limiteremo a utilizzare la classe RecursiveCharacterTextSplitter per l'estrazione di chunk coerenti dalle fonti di riferimento, preservando la semantica delle informazioni originali.

2.2.3 Hugging Face

Hugging Face [11] è una piattaforma open source dedicata all'intelligenza artificiale e al machine learning. Grazie alla libreria Model Hub è possibile il download di modelli già addestrati per svariati scopi, tra cui quelli di cui usufruiremo ai fini dello sviluppo del progetto.

⁵ API - Application Programming Interface: un insieme di regole e protocolli che permettono a due applicazioni di comunicare correttamente.

⁶ endpoint: nell'ambito delle API, un URL specifico tramite cui un'applicazione riceve ed elabora richieste API

2.2.3.1 FastEmbed

FastEmbed [7] è una libreria Python adatta a generare embedding di testi e immagini. Come già accennato in precedenza l'embedding permette di ottenere un vettore di reali che mantengano la semantica linguistica, nel caso di testo, del dato originale.

Tra gli innumerevoli modelli offerti da FastEmbed si è scelto di utilizzare `intfloat/multilingual-e5-small` come bi-encoder, il cui download è possibile grazie alla Model Hub di Hugging Face.

2.2.3.2 SentenceTransformers

SentenceTransformers [9] è un framework ideato per la computazione di embedding e l'utilizzo di modelli di reranking. Utilizzeremo quest'ultimo per poter disporre di `ms-marco-MiniLM-L-6-v2`, un cross-encoder adatto ai nostri scopi. Anche in questo caso il download del modello avverrà automaticamente tramite Hugging Face durante l'inizializzazione della nostra applicazione.

2.2.4 Ollama e llama.cpp

Ollama [5] è un framework open source concepito per semplificare l'esecuzione, la configurazione e la gestione locale degli LLM su sistemi operativi come macOS, Windows e Linux.

Il framework espone un set di API RESTful standard che consente di interagire programmaticamente con i modelli scaricati. Grazie al supporto di librerie ufficiali e sviluppate dalla community per linguaggi come Python e JavaScript, è possibile integrare le capacità di ragionamento degli LLM all'interno di applicazioni, script o flussi di lavoro aziendali in modo estremamente semplice.

Ollama offre, oltretutto, un'immagine Docker ufficiale da eseguire in un opportuno container. Maggiori informazioni su Docker e sul concetto di container verranno fornite nella relativa sezione di questo capitolo.

Si noti per di più che, poiché i modelli sono eseguiti interamente in locale, il sistema non trasmette alcuna informazione a server esterni, garantendo massima privacy e totale indipendenza da connessioni internet o provider terzi.

Inoltre, giacché l'elaborazione grava esclusivamente sull'hardware dell'utente, il costo marginale per ogni singola query posta all'LLM diviene fondamentalmente nullo.

In aggiunta a ciò, il framework supporta e aggiorna costantemente una libreria di modelli open weights, tra cui Llama, Mistral, Gemma e Phi-3, permettendo di selezionare l'architettura più idonea in base al caso d'uso specifico.

Per permettere a reti neurali complesse di funzionare su hardware consumer, Ollama implementa un processo noto come quantizzazione dei modelli. Questa tecnica riduce la precisione matematica dei pesi dell'LLM, abbattendo pertanto in maniera drastica la quantità di RAM nonché la potenza di calcolo richieste per l'esecuzione del modello.

Tuttavia, questo approccio comporta alcuni trade-off inevitabili. In primis, a causa dei limiti hardware dei dispositivi consumer, i modelli eseguiti localmente presentano un numero di parametri inferiore e, conseguentemente, una qualità dell'output e capacità di ragionamento più limitate.

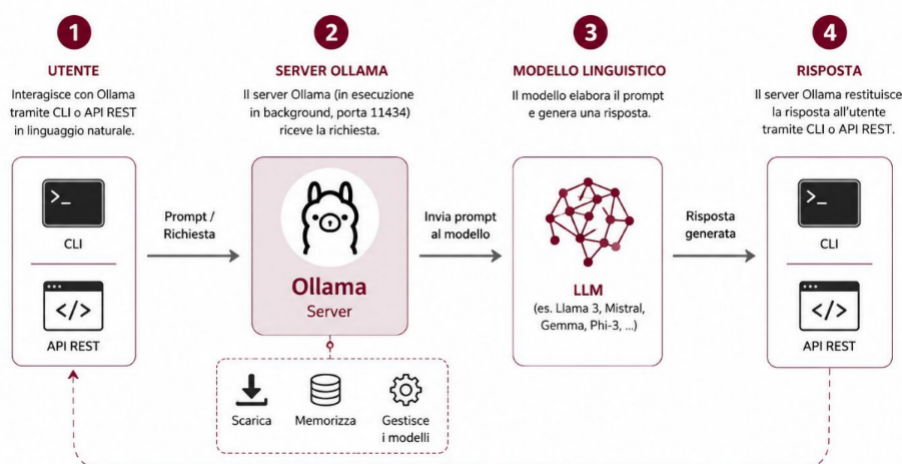


Figura 2.2. Diagramma di flusso di un prompt effettuato a un LLM tramite Ollama.

In secondo luogo, l'effettiva velocità di esecuzione di Ollama è rigidamente vincolata all'hardware della macchina ospitante, prediligendo architetture basate su GPU a quelle CPU. Naturalmente, l'esecuzione di un LLM tramite Ollama è comunque possibile su macchine sprovviste di GPU, a patto di accettare un'inferenza più lenta del modello.

L'obiettivo di Sapienza-DC consisterà difatti nel permettere a qualsiasi utente, dotato di una scheda video o anche semplicemente del proprio processore, di eseguire il progetto localmente, fornendo la possibilità di affidarsi, in caso di necessità, anche ad API esterne, qualora l'hardware della propria macchina dovesse risultare non sufficientemente performante. Entreremo maggiormente nei dettagli di questa tematica nei capitoli successivi.

A tal proposito, vedremo come sia possibile utilizzare direttamente llama.cpp per gestire manualmente il caricamento, la gestione nonché l'esecuzione di query verso LLM locali, con lo scopo di ridurre l'overhead introdotto da Ollama, il quale non è altro che un wrapper di quest'ultimo.

Brevemente, llama.cpp è un'implementazione in linguaggio C/C++ del codice di inferenza dei modelli LLaMA (Large Language Model Meta AI), successivamente esteso a molteplici altre architetture, il cui scopo fondamentale consiste nel massimizzare la velocità di esecuzione locale di LLM. Rimuovendo l'overhead dovuto a Ollama e accettando la maggiore difficoltà implementativa che consegue da questa scelta, saremo in grado di minimizzare i requisiti hardware richiesti per eseguire modelli localmente, soprattutto su macchine sprovviste di GPU.

2.2.5 SearXNG

Utilizzato in una prima versione del progetto, SearXNG [3] è un metamatore di ricerca che aggrega risultati da svariati servizi di ricerca e database⁷. Garantisce anonimata e privacy per gli utenti che lo utilizzano, i quali non subiscono tracking né profilazione.

⁷ database: base di dati, una raccolta organizzata di informazioni o dati strutturati.

2.2.6 Crawl4AI

Crawl4AI [4] è una libreria Python open source impiegata principalmente per estrarre e ripulire dati in modo strutturato e automatico da pagine web.

In realtà, Sapienza-DC non utilizza direttamente questa libreria, ma fa affidamento a un secondo progetto precedentemente sviluppato, MWP (Minerva Web Parser), per effettuare scraping e parsing di pagine web mediante Crawl4AI. Maggiori informazioni sul funzionamento di MWP verranno presentate nei capitoli successivi.

Minerva Web Parser è stato realizzato come progetto di esonero (al quale si accennava nell'introduzione di questo elaborato) per il corso di Laboratorio di Ingegneria Informatica, ed è stato successivamente aggiornato per includere nuove funzionalità durante lo sviluppo di Sapienza-DC stesso.

2.2.7 Jinja2

Jinja2 [10] è un engine di templating utilizzato per definire pagine web generiche contenenti opportuni segnaposto da popolare a runtime tramite chiamate API al backend.

Verrà utilizzato per implementare un frontend in grado di servire, previa ottenimento di dati dal server API, pagine HTML (HyperText Markup Language) dinamiche.

2.2.8 TailwindCSS

TailwindCSS [14] è un framework CSS⁸ open source per lo sviluppo di UI moderne ed eleganti. Tailwind mette a disposizione una serie di classi CSS dette di utility da combinare e utilizzare insieme ad altre classi per definire lo stile di ciascun elemento in modo intuitivo e modulare.

2.2.8.1 Utility class

Le utility classes rappresentano l'approccio odierno allo styling CSS: l'idea è quella di avere numerose classi che svolgono compiti ben precisi, come, ad esempio, l'applicazione di una determinata proprietà di stile a un elemento HTML.

2.3 Containerizzazione ed orchestrazione

La containerizzazione è una tecnologia che consente di racchiudere un'applicazione, assieme alle sue dipendenze, in un'unità standardizzata detta container. I container permettono agli sviluppatori di garantire:

- **portabilità** - affinché l'applicazione possa essere eseguita correttamente a prescindere dall'ambiente o dalla piattaforma su cui viene eseguito il container;
- **isolamento** - ciascun container opera indipendentemente dagli altri, in modo tale da evitare possibili interferenze;

⁸ CSS - Cascading Style Sheets: linguaggio utilizzato per definire la formattazione e il layout di documenti HTML.

- **efficienza** - i container condividono il kernel del sistema operativo della macchina sulla quale vengono eseguiti e sono pertanto più leggeri delle macchine virtuali, garantendo dunque un minore overhead computazionale.

L'orchestrazione di container consiste invece nel processo di automazione della gestione di questi ultimi, al fine di sviluppare applicazioni su larga scala. Un'applicazione può difatti sfruttare molteplici container per implementare diverse funzionalità in modo incredibilmente modulare.

L'impiego di container separati permette inoltre di evitare problematiche relative a SPOF (Single Point Of Failure). Se un servizio non dovesse risultare disponibile sarà il singolo container a non essere raggiungibile, mentre gli altri servizi offerti dall'app rimarranno online, purché non dipendenti dal container momentaneamente fuori uso.

2.3.1 Architettura a microservizi: Docker e Docker Compose

L'approccio sopra descritto prende il nome di architettura a microservizi. Ribadiamo sinteticamente l'intuizione alla base di quest'ultimo: dividere le funzionalità offerte dalla nostra applicazione in una serie di microservizi ospitati in altrettanti container, opportunamente orchestrati.

Al fine di implementare quanto appena visto nel progetto, faremo uso di Docker [1], una piattaforma open source per la distribuzione automatizzata di applicazioni all'interno di container isolati.

Docker fornisce un ambiente isolato, riproducibile e portabile, pensato per l'esecuzione nonché la distribuzione di un progetto software in un ambiente di testing o di produzione.

Per orchestrare ciascun container viene inoltre reso disponibile Docker Compose, uno strumento per la gestione di applicazioni multi-container basato su file YAML⁹.

2.4 Database relazionale

Un database relazionale è un particolare tipo di base di dati in cui i dati sono organizzati sotto forma di righe e di colonne. Ciascuna tabella della base di dati rappresenta una determinata relazione $\mathcal{R} \subseteq S_1 \times S_2 \times \dots \times S_n$, cioè un sottoinsieme del prodotto cartesiano di n insiemi. Ciascuna tupla $t \in \mathcal{R}$ sarà nella forma $t = (s_1, s_2, \dots, s_n)$, dove $s_1 \in S_1, s_2 \in S_2, \dots, s_n \in S_n$. Nella pratica, ogni riga della tabella consisterà in una n -upla di elementi che assumono valori in un nel dominio (un intero, una stringa. etc.) della corrispondente colonna, detta attributo o campo. Si noti infine che, essendo $\mathcal{R}$ un insieme, $\neg \exists (t, \hat{t}) \in \mathcal{R} \mid t = \hat{t}$, cioè $\mathcal{R}$ non ammette tuple duplicate.

2.4.1 SQL

SQL (Structured Query Language) è il linguaggio utilizzato per interagire programmaticamente con un RDBMS (Relational DataBase Management System), al fine di

⁹ YAML - Yet Another Markup Language: un formato di serializzazione di dati basato su linguaggio naturale.

aggiungere, modificare, eliminare o ricercare informazioni all'interno delle tabelle della base di dati (DML¹⁰); gestire la struttura del database e delle tabelle che compongono quest'ultimo (DDL¹¹); la concessione e la revoca di permessi relativi alla base di dati (DCL¹²); controllo e gestione delle transazioni, ossia insiemi di operazioni che devono essere eseguite in modo atomico (TCL¹³).

2.4.2 MariaDB

MariaDB [2] è un sistema di gestione di basi di dati relazionali open source. In particolare, non è altro che un fork di MySQL [13]. Progettato per garantire compatibilità con MySQL stesso, MariaDB offre un'immagine Docker per la gestione l'interazione con database mediante container.

2.5 Metodologia di sviluppo agile

La metodologia agile consiste in un approccio allo sviluppo basato sulla distribuzione di software implementati secondo un processo rapido e iterativo. Contrapponendosi ai modelli tradizionali di sviluppo del software, come ad esempio il modello a cascata (waterfall model¹⁴), la metodologia agile propone un approccio meno strutturato e incentrato sulla consegna ripetuta nel tempo di diversi MVP¹⁵.

Questi ultimi vengono sottoposti, volta per volta, ad analisi da parte dei clienti nonché degli sviluppatori con l'obiettivo di rifinire l'approccio utilizzato, modificando anche completamente la soluzione proposta qualora ritenuto necessario, per il raggiungimento di un prodotto migliore nella successiva iterazione. Si tratta di una metodologia frequentemente impiegata in progetti dinamici e adattivi, come, ad esempio, Sapienza-DC.

¹⁰ DML: Data Manipulation Language

¹¹ DDL: Data Definition Language

¹² DCL: Data Control Language

¹³ TCL: Transaction Control Language

¹⁴ waterfall model: metodologia di sviluppo software basata sulla definizione precisa e strutturata di un insieme di requisiti iniziali da rispettare, seguita da un'implementazione prevedibile e lineare di questi ultimi.

¹⁵ MVP - Minimum Viable Product: una versione basilare di un prodotto o servizio volta ad offrire solo le funzionalità essenziali richieste dal cliente.

Capitolo 3

Progettazione dell'applicazione

Avendo ora a disposizione le conoscenze preliminari necessarie a comprendere l'impiego dei principali strumenti utilizzati per sviluppare Sapienza-DC, è bene effettuare, prima dell'implementazione vera e propria dell'applicativo, alcune dovute considerazioni riguardo la progettazione delle componenti fondamentali che andranno a costituire l'architettura finale del progetto.

Si noti che, come già menzionato, la metodologia di sviluppo agile non prevede obbligatoriamente una raccolta estensiva di requisiti (se non le principali funzionalità ricercate nell'applicativo) né tantomeno la realizzazione iniziale di diagrammi e modelli tipici dell'approccio a cascata. Difatti, l'assenza di requisiti altamente specifici richiede uno sviluppo dinamico e soggetto a continui cambiamenti, motivo per cui i diagrammi che proporremo di seguito rispecchiano il risultato di quanto ottenuto a fine progetto.

3.1 Diagramma delle classi UML

Un diagramma delle classi UML (*Unified Modeling Language*) è un diagramma strutturale utilizzato per definire le principali classi, i loro attributi e le relazioni che legano queste ultime. Durante l'esecuzione del software una classe può essere istanziata sotto forma di uno o più oggetti, che definiscono il livello estensionale di quest'ultima.

Nella programmazione orientata agli oggetti, il diagramma delle classi diviene uno strumento particolarmente utile per disporre di una visione complessiva riguardo il funzionamento del sistema nonché delle relazioni che sussistono tra le varie componenti che lo costituiscono.

A puro scopo nozionistico, si noti che ciascuna classe nel diagramma è contraddistinta da un nome, una serie di attributi locali, ciascuno con il proprio tipo, e da una serie di operazioni associate ad essa.

Per evitare di appesantire il diagramma UML che presenteremo di seguito, si è deciso di non includere nelle specifiche delle operazioni di ciascuna classe i tipi dei parametri in input, così come si è preferito sostituire con la dicitura (...) i parametri accettati dalle operazioni, qualora ve ne fossero in numero elevato.

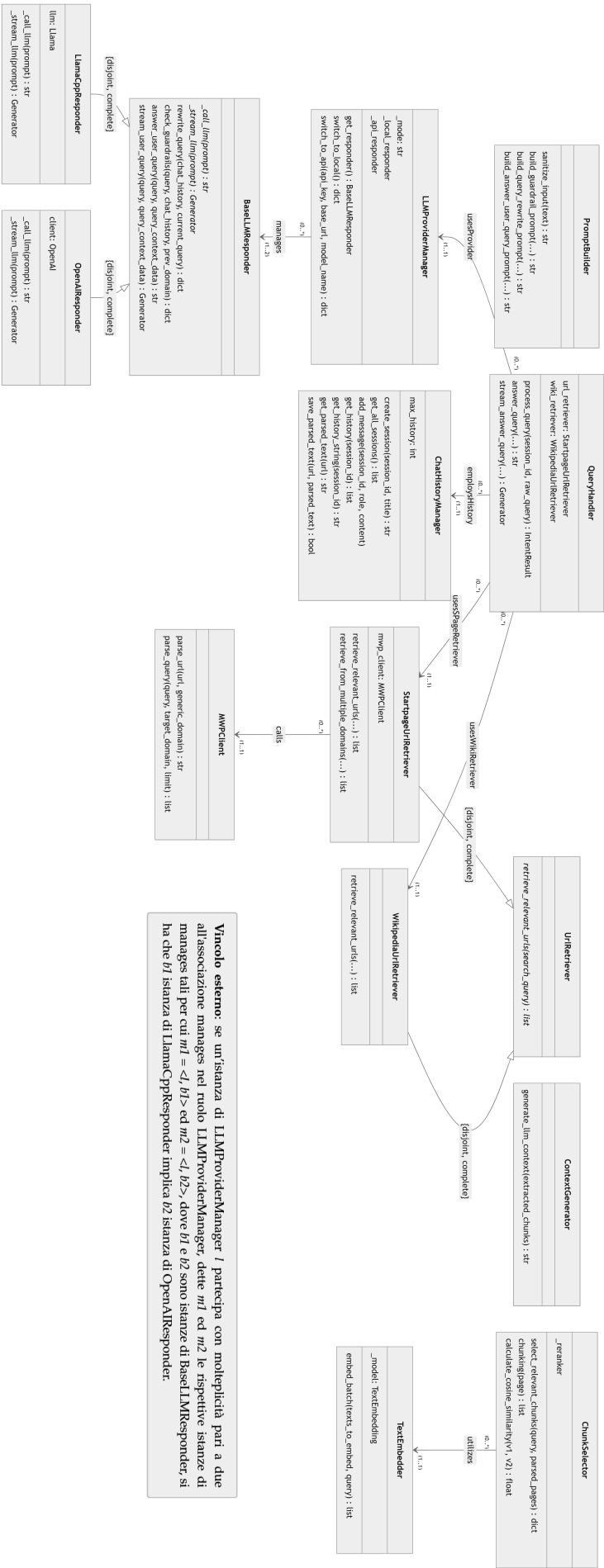


Figura 3.1. Diagramma delle classi UML del progetto.

3.2 Modello ER

Il modello Entità-Relazione (ER) è un modello di dati impiegato per la progettazione concettuale di basi di dati. La rappresentazione di tali schemi concettuali avviene tipicamente in modo grafico, mediante diagrammi Entità-Relazione. Ogni schema descrive l'aspetto intensionale dei dati; a uno stesso schema possono corrispondere, nel tempo, più istanze, le quali ne costituiscono invece l'aspetto estensionale.

Analogamente a quanto illustrato per i diagrammi delle classi UML, anche il modello ER introduce costrutti simili. Tra questi figurano le entità (analogo delle classi UML), gli attributi di entità, le relazioni (o associazioni) a cui ciascuna entità partecipa con il rispettivo ruolo, gli attributi di relazione, le gerarchie di generalizzazione e specializzazione (IS-A), nonché alcuni vincoli di integrità caratteristici del modello Entità-Relazione.

Come per il diagramma delle classi UML, anche per lo schema concettuale del modello ER, ottenuto mediante la corrispettiva progettazione concettuale, verrà fornito direttamente nella sua versione finale, ossia quanto ottenuto al termine dell'implementazione di Sapienza-DC.

3.2.1 Progettazione concettuale

La progettazione concettuale prevede la traduzione dei requisiti forniti nel relativo schema concettuale del modello ER. Nel nostro caso, poiché l'applicazione non deve gestire quantità ingenti di dati, ma solo tenere traccia delle conversazioni e dei messaggi inviati dall'utente, nonché garantire il salvataggio dei dati estratti dalle pagine web (identificate tramite URL), ai fini di *caching*¹, lo schema concettuale risultante è piuttosto semplice.

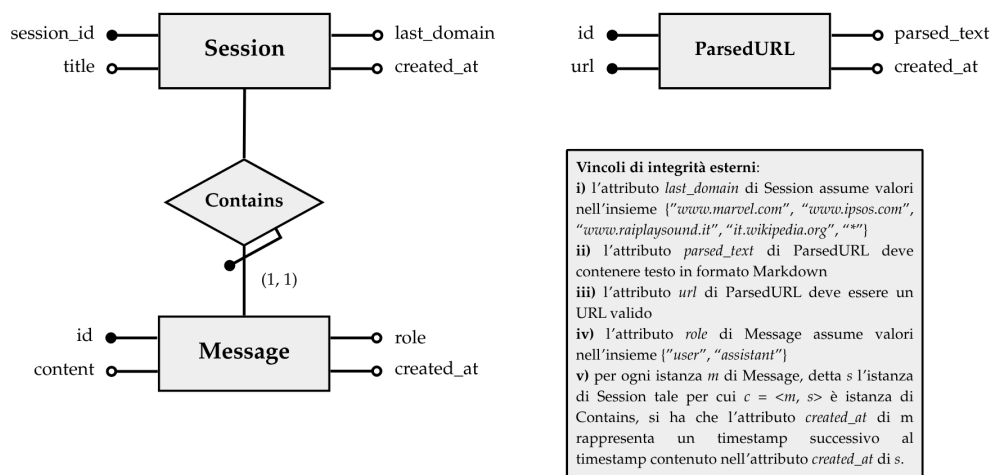


Figura 3.2. Schema concettuale del modello ER.

¹ caching: processo di archiviazione temporanea di copie di dati, effettuato con lo scopo di garantire latenza minore in fase di recupero delle informazioni.

3.2.2 Progettazione logica

La progettazione logica permette di tradurre lo schema concettuale ottenuto al termine della fase di progettazione concettuale in uno schema relazionale per la nostra base di dati.

Un accorgimento di vitale importanza da tenere presente in questa fase della progettazione è il seguente: si vuole evitare ad ogni costo la perdita di informazioni nel passaggio da uno schema all'altro, affinché anche lo schema relazionale continui a rispettare i requisiti imposti inizialmente.

3.2.2.1 Ristrutturazione dello schema concettuale

La prima fase della progettazione logica prevede la ristrutturazione dello schema concettuale precedentemente ottenuto. Essa si articola nei seguenti sei passaggi:

1. analisi delle ridondanze;
2. eliminazione degli attributi multivalore;
3. eliminazione degli attributi composti;
4. eliminazione delle IS-A e di eventuali generalizzazioni;
5. scelta degli identificatori principali di entità e relazioni;
6. specifica degli ulteriori vincoli esterni.

Giacché lo schema concettuale da ristrutturare è elementare, l'unico passo applicabile è il quinto. In particolare, si sceglie come identificatore principale dell'entità ParsedURL l'attributo *id*. Le rimanenti scelte sono immediate e non vengono dunque esplicitate. Lo schema ristrutturato appare pertanto fondamentalmente invariato, così come i vincoli esterni definiti in precedenza.

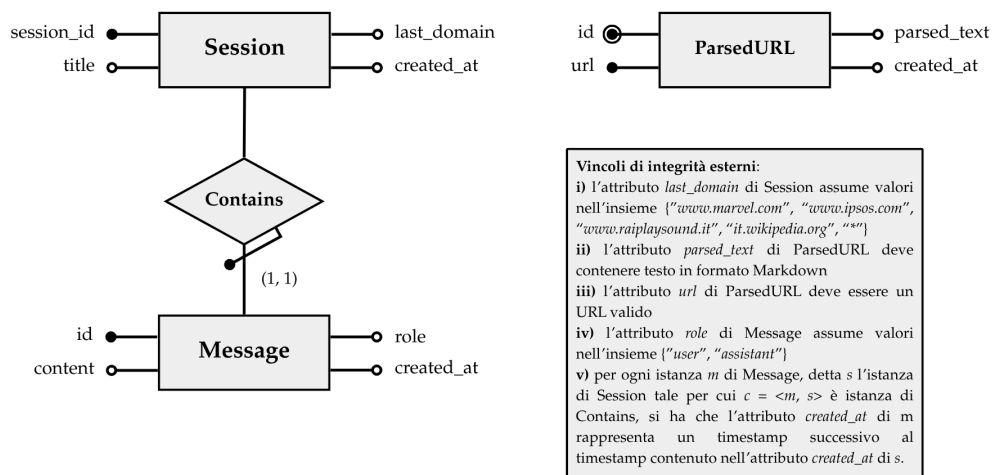


Figura 3.3. Schema concettuale ristrutturato.

3.2.2.2 Traduzione diretta

Lo scopo della traduzione diretta è quello di trasformare lo schema concettuale ristrutturato in uno schema relazionale. Ciascuna relazione viene rappresentata secondo la seguente notazione:

$$\mathcal{R}(\underline{\mathcal{A}_1}, \underline{\mathcal{A}_2}, \mathcal{A}_3, \mathcal{A}_4^*, \dots, \mathcal{A}_n)$$

dove gli attributi sottolineati indicano la chiave primaria della relazione; gli attributi segnati con l'asterisco attributi per cui è ammesso il valore nullo, contrariamente agli attributi non contrassegnati mediante quest'ultimo.

Altre informazioni essenziali come eventuali vincoli di *foreign key*², inclusioni, vincoli di chiave o ulteriori vincoli di tupla o inter-tupla sono da definire al di sotto della relazione stessa.

Una volta terminata la traduzione diretta, otteniamo il seguente schema $\mathcal{S}$ per la nostra base di dati $\mathcal{B}$:

```

Session(session_id, title*, last_domain*, created_at)
  vincolo di tupla: title is NULL  $\Rightarrow$  title = "New Chat"
  vincolo di tupla: last_domain is NULL  $\Rightarrow$  last_domain = ""
  vincolo di tupla: last_domain is not NULL  $\Rightarrow$  last_domain  $\in$  {"www.marvel.com",
    "www.ipsos.com", "www.raiplaysound.com", "it.wikipedia.org",
    "*"}
Message(id, role, content, created_at)
  foreign key: Message[id]  $\subseteq$  Contains[message]
  vincolo di tupla: role  $\in$  {"user", "assistant"}
ParsedURL(id, url, parsed_text*, created_at)
  chiave: url
  vincolo esterno: parsed_text is not NULL  $\Rightarrow$  parsed_text is in MD format
  vincolo esterno: url is a valid URL
Contains(message, session)
  foreign key: Contains[message]  $\subseteq$  Message[id]
  foreign key: Contains[session]  $\subseteq$  Session[session_id]
  vincolo esterno:  $(m, s) \in \text{Contains} \Rightarrow \text{timestamp}(m) > \text{timestamp}(s)$ 

```

Si noti infine che si è scelto di assegnare, per comodità, opportuni valori di default agli attributi *title* e *last_domain* della tabella Session.

3.2.2.3 Ristrutturazione dello schema logico

L'ultima fase della progettazione logica prevedere una ristrutturazione dello schema relazionale per motivi di efficienza. Ai fini della nostra applicazione, è di interesse conoscere, ogni qualvolta si accede a un messaggio m , la sessione s a cui esso appartiene.

² foreign key: è un vincolo di integrità referenziale definito fra una sequenza non vuota $\mathcal{X}$ di n attributi di una relazione $\mathcal{R}_1$ ed una sequenza $\mathcal{Y}$ di n attributi che formano una chiave di una relazione $\mathcal{R}_2$; è un'asserzione che impone che per ogni istanza $\mathcal{B}$ della base di dati ogni combinazione di valori su $\mathcal{X}$ che non contengono NULL e che sono presenti nella istanza di $\mathcal{R}_1$ in $\mathcal{B}$ compaia come combinazione di valori su $\mathcal{Y}$ nella istanza di $\mathcal{R}_2$ in $\mathcal{B}$.

Notando a questo punto che le tabelle Message e Contains sono fortemente accoppiate, si predispose l'accorpamento di Contains a Message, ottenendo il seguente schema relazionale finale per la nostra base di dati:

Session(session_id, title*, last_domain*, created_at)
 vincolo di tupla: title is NULL $\Rightarrow$ title = "New Chat"
 vincolo di tupla: last_domain is NULL $\Rightarrow$ last_domain = ""
 vincolo di tupla: last_domain is not NULL $\Rightarrow$ last_domain $\in$ {"www.marvel.com", "www.ipsos.com", "www.raiplaysound.com", "it.wikipedia.org", "*"}
Message(id, session, role, content, created_at)
 foreign key: Message[session] $\subseteq$ Session[session_id]
 vincolo di tupla: role $\in$ {"user", "assistant"}
 vincolo esterno: $m \in \text{Message} \Rightarrow m.\text{created_at} > \text{timestamp}(m.\text{session})$
ParsedURL(id, url, parsed_text*, created_at)
 chiave: url
 vincolo esterno: parsed_text is not NULL $\Rightarrow$ parsed_text is in MD format
 vincolo esterno: url is a valid URL

La progettazione della nostra base di dati è a questo punto ultimata. Per completezza, si fornisce di seguito il codice SQL per la definizione delle tabelle del database definitivo.

```

1 CREATE TABLE IF NOT EXISTS sessions (
2     session_id VARCHAR(36) PRIMARY KEY,
3     title VARCHAR(255) DEFAULT 'New Chat',
4     last_domain VARCHAR(255) DEFAULT '',
5     created_at DATETIME DEFAULT CURRENT_TIMESTAMP
6 );
7
8 CREATE TABLE IF NOT EXISTS messages (
9     id INT AUTO_INCREMENT PRIMARY KEY,
10    session_id VARCHAR(36) NOT NULL,
11    role VARCHAR(20) NOT NULL,
12    content TEXT NOT NULL,
13    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
14    FOREIGN KEY (session_id) REFERENCES sessions(session_id) ON DELETE
15    CASCADE
16 );
17
18 CREATE TABLE IF NOT EXISTS parsed_urls (
19     id INT AUTO_INCREMENT PRIMARY KEY,
20     url TEXT NOT NULL UNIQUE,
21     parsed_text LONGTEXT,
22     created_at DATETIME DEFAULT CURRENT_TIMESTAMP
23 );

```

Listato 3.1. Codice SQL per la creazione delle tabelle della base di dati.

Si noti infine l'aggiunta della clausola ON DELETE CASCADE, la quale pre-dispone l'eliminazione automatica e coerente di tutti i messaggi associati a una determinata sessione nel caso in cui quest'ultima dovesse essere eliminata. Questa scelta permette di mantenere, in ogni istante, l'integrità referenziale del database.

Capitolo 4

Architettura del sistema

Nel seguente capitolo tratteremo i principali dettagli inerenti all'architettura di Sapienza-DC. Come già anticipato nel secondo capitolo, il progetto adotta un'architettura a microservizi containerizzata tramite Docker e orchestrata mediante Docker Compose.

I microservizi che costituiscono l'applicazione sono tre e consistono sommariamente in un backend per la gestione nonché elaborazione lato server delle chiamate API, un frontend che agisce da *proxy*¹ API per l'inoltro di richieste lato client al server API, servendo e popolando dinamicamente pagine HTML tramite templating Jinja2 e JS; un database basato su MariaDB per la gestione e la persistenza dei dati dell'applicativo. Naturalmente, ciascun microservizio viene eseguito su un container dedicato al fine di garantire modularità, isolamento e portabilità del progetto.

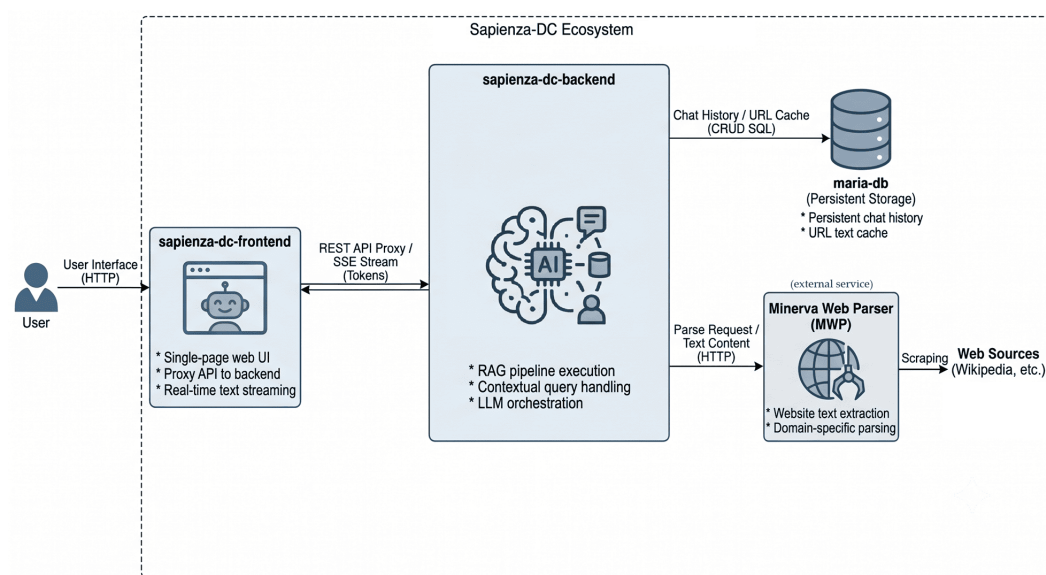


Figura 4.1. Diagramma architetturale di Sapienza-DC.

In realtà, l'applicazione non potrebbe operare correttamente senza un quarto microservizio, esterno al progetto, al quale il backend effettua a sua volta richieste

¹ who is that?

di parsing.

Tale microservizio coincide difatti nell'API server di Minerva Web Parser, il quale espone alcune API essenziali per la nostra pipeline RAG. In quanto progetto da noi stessi sviluppato, tratteremo brevemente anche alcuni dettagli architetturali e implementativi di MWP.

Per ognuno dei microservizi sopra elencati descriveremo dunque nel dettaglio i principali moduli che li costituiscono, nonché le relative funzionalità offerte.

4.1 Architettura lato server (backend)

Il backend costituisce il cuore logico dell'applicazione e si occupa di gestire, tra le altre cose, l'esposizione delle API RESTful per il frontend; il salvataggio, l'inserimento e l'eliminazione dei dati nel database MariaDB; l'interazione nonché interfacciamento con API remote o modelli locali per l'impiego di LLM, opportunamente istruiti mediante tecniche di *prompt engineering*²; il processamento e la valutazione della query dell'utente; la corretta attuazione della pipeline di Retrieval-Augmented Generation; il reperimento dinamico di informazioni rilevanti alla query da pagine web tramite MWP, prevedendo inoltre opportuni meccanismi di *fallback*³. In particolare, i moduli in cui si articola sono i seguenti:

- **api_server**: consta di un'applicazione esposta tramite FastAPI; espone un insieme di endpoint RESTful atti a gestire le sessioni dell'utente, la selezione dinamica dell'LLM, il routing della query ricevuta e in generale il flusso di esecuzione della pipeline RAG; grazie all'utilizzo del protocollo SSE⁴ è inoltre in grado di realizzare lo streaming dei token testuali generati dall'LLM, insieme a messaggi inerenti allo stato corrente di avanzamento della pipeline stessa.
- **db_manager**: come intuibile dal nome alquanto evocativo, questo modulo si occupa della connessione e dell'accesso, da parte del backend, alla base di dati del nostro applicativo, oltre all'esecuzione delle principali operazioni previste sulle tabelle presenti nel DB⁵.
- **llm_manager**: implementa e coordina l'interazione con i modelli linguistici, astruendo quest'ultima rispetto al paradigma di computazione sottostante, sia essa eseguita localmente oppure su un server remoto acceduto attraverso chiamate API, compatibili con OpenAI⁶. Ulteriore compito di questo modulo consiste nella generazione dei prompt per gli LLM, su cui ci soffermeremo maggiormente nel capitolo che segue.
- **query_handler**: modulo incaricato del processamento iniziale della query ricevuta dall'utente, il controllo della validità nonché adeguatezza di quest'ultima, la chiamata ai moduli di scraping necessari ad effettuare la ricerca di URL

² cate<3

³ sos

⁴ miaomiao

⁵ bruh

⁶ sas

di pagine web contenenti informazioni presumibilmente rilevanti alla query nonché la generazione della risposta alla medesima.

- **rag**: estrae, attraverso opportune tecniche algoritmiche che approfondiremo in fase di implementazione della pipeline RAG, le informazioni utili a rispondere alla query dell'utente a partire dai dati ricercati sul web. Il paradigma impiegato è noto come *Two-Stage Retrieval* e consiste, come sappiamo, nell'utilizzo di un bi-encoder e di un cross-encoder in successione per stilare una classifica, ordinata in senso decrescente, delle informazioni più semanticamente pertinenti rispetto alla query q . Oltre a ciò, questo modulo si interfaccia con il servizio esterno MWP per reperire, dato un insieme $\mathcal{K}$ di k URL di pagine web, le informazioni contenute in ciascuna di esse in forma testuale.
- **url_retriever**: reperisce dinamicamente i primi k URL rilevanti a partire dalla query dell'utente, preliminarmente riscritta in un formato adatto per la ricerca su un *search engine*⁷. Al fine di minimizzare il numero di eventualità in cui tale ricerca dovesse fallire, il modulo impiega un utile meccanismo di fallback, le cui specifiche introdurremo nel prossimo capitolo.

4.2 Architettura lato client (frontend)

Il frontend è un microservizio leggero, progettato per inoltrare le richieste al backend di Sapienza-DC e fornire al contempo un'interfaccia utente moderna e intuitiva. Esso è costituito da un unico modulo **src** che si occupa della realizzazione delle funzionalità sopra descritte. Concretamente, quest'ultimo garantisce quanto segue:

- la gestione efficiente del *rendering*⁸ delle pagine web dell'applicativo, usufruendo dei template di Jinja2 e delle classi utility di TailwindCSS per offrire una UI elegante e *responsive*⁹.
- l'inoltro tempestivo delle richieste del client browser verso il backend tramite API proxy e il controllo asincrono dei flussi SSE provenienti da quest'ultimo. Ciascun token ricevuto viene utilizzato per aggiornare progressivamente l'interfaccia utente, in modo tale da fornire un'esperienza di utilizzo fluida e reattiva. Si noti che qualora non avessimo utilizzato questo approccio, l'utente avrebbe dovuto attendere diversi secondi (o minuti in caso di inferenza locale su CPU), senza alcun riscontro visivo, il processamento della propria query e la generazione della risposta finale. L'aggiornamento in tempo reale dello stato attuale della pipeline, nonché lo streaming token per token della risposta dell'LLM rassicura pertanto l'utente riguardo al corretto funzionamento dell'applicazione e all'effettiva elaborazione della richiesta inoltrata.

⁷ cos'è un se?

⁸ cos'è rendering

⁹ brruuu

4.3 Architettura per la persistenza dei dati (MariaDB)

Come ormai noto, il database basato su MariaDB rappresenta un microservizio a sé stante ospitato nel rispettivo container Docker, ove sono ospitate le tre tabelle **sessions**, **messages** e **parsed_urls** definite in fase di progettazione. Gli obiettivi della base di dati dovrebbero essere a questo punto lampanti, ma li riassumiamo nuovamente per meri fini di completezza della trattazione:

- la gestione della cronologia della conversazioni con l'utente associate a una determinata sessione; questo garantisce che il chatbot sia in grado di mantenere il contesto dei messaggi scambiati in precedenza, in modo tale da sostenere un dialogo coerente e logico;
- l'indicizzazione e la memorizzazione (caching) di URL di cui è già stato fatto scraping per minimizzare la latenza nel recupero dei dati dalle fonti selezionate ed evitare problematiche relative a *rate limiting*¹⁰. Qualora lo stesso URL dovesse essere selezionato come rilevante per rispondere a una query successiva dell'utente, il backend procederà a recuperare le informazioni già estratte direttamente dal DB, abbattendo il tempo di attesa da parte dell'utente e riducendo per di più il carico computazionale della pipeline RAG, giacché il processo di parsing è intrinsecamente costoso.

4.4 Architettura di integrazione ed estrazione dal web (MWP)

¹⁰ cos

Capitolo 5

Implementazione della pipeline RAG

Fase implementativa.

Capitolo 6

Ottimizzazioni, sicurezza e benchmark

Capitolo 7

Conclusione

Bibliografia

- [1] Inc. Docker. Docker documentation. <https://docs.docker.com/>, 2026. Ultimo accesso: 01 Giugno 2026.
- [2] MariaDB Foundation. Mariadb documentation. <https://mariadb.com/docs>, 2026. Ultimo accesso: 01 Giugno 2026.
- [3] Markus Heiser. Searxng documentation. <https://docs.searxng.org/>, 2026. Ultimo accesso: 01 Giugno 2026.
- [4] Hossein. Crawl4ai documentation. <https://docs.crawl4ai.com/>, 2026. Ultimo accesso: 01 Giugno 2026.
- [5] Michael Chiang Jeffrey Morgan. Ollama documentation. <https://ollama.readthedocs.io/en/>, 2026. Ultimo accesso: 01 Giugno 2026.
- [6] Fabio Pastore. Repository ufficiale di sapienza-dc. <https://github.com/fabio-pastore/sapienza-DC>, 2026. Ultimo accesso: 01 Giugno 2026.
- [7] Qdrant. Fastembed documentation. <https://qdrant.tech/documentation/fastembed/>, 2026. Ultimo accesso: 01 Giugno 2026.
- [8] Sebastián Ramírez. Fastapi documentation. <https://fastapi.tiangolo.com/>, 2026. Ultimo accesso: 01 Giugno 2026.
- [9] Nils Reimers. Sentencetransformers documentation. <https://sbert.net/>, 2026. Ultimo accesso: 01 Giugno 2026.
- [10] Armin Ronacher. Jinja2 documentation. <https://jinja.palletsprojects.com/en/stable/>, 2026. Ultimo accesso: 01 Giugno 2026.
- [11] Hugging Face Team. Hugging face documentation. <https://huggingface.co/docs>, 2026. Ultimo accesso: 01 Giugno 2026.
- [12] LangChain Team. Langchain documentation. <https://docs.langchain.com/>, 2026. Ultimo accesso: 01 Giugno 2026.
- [13] MySQL Documentation Team. Mysql documentation. <https://dev.mysql.com/doc/>, 2026. Ultimo accesso: 01 Giugno 2026.
- [14] Adam Wathan. Tailwindcss documentation. <https://v2.tailwindcss.com/docs>, 2026. Ultimo accesso: 01 Giugno 2026.